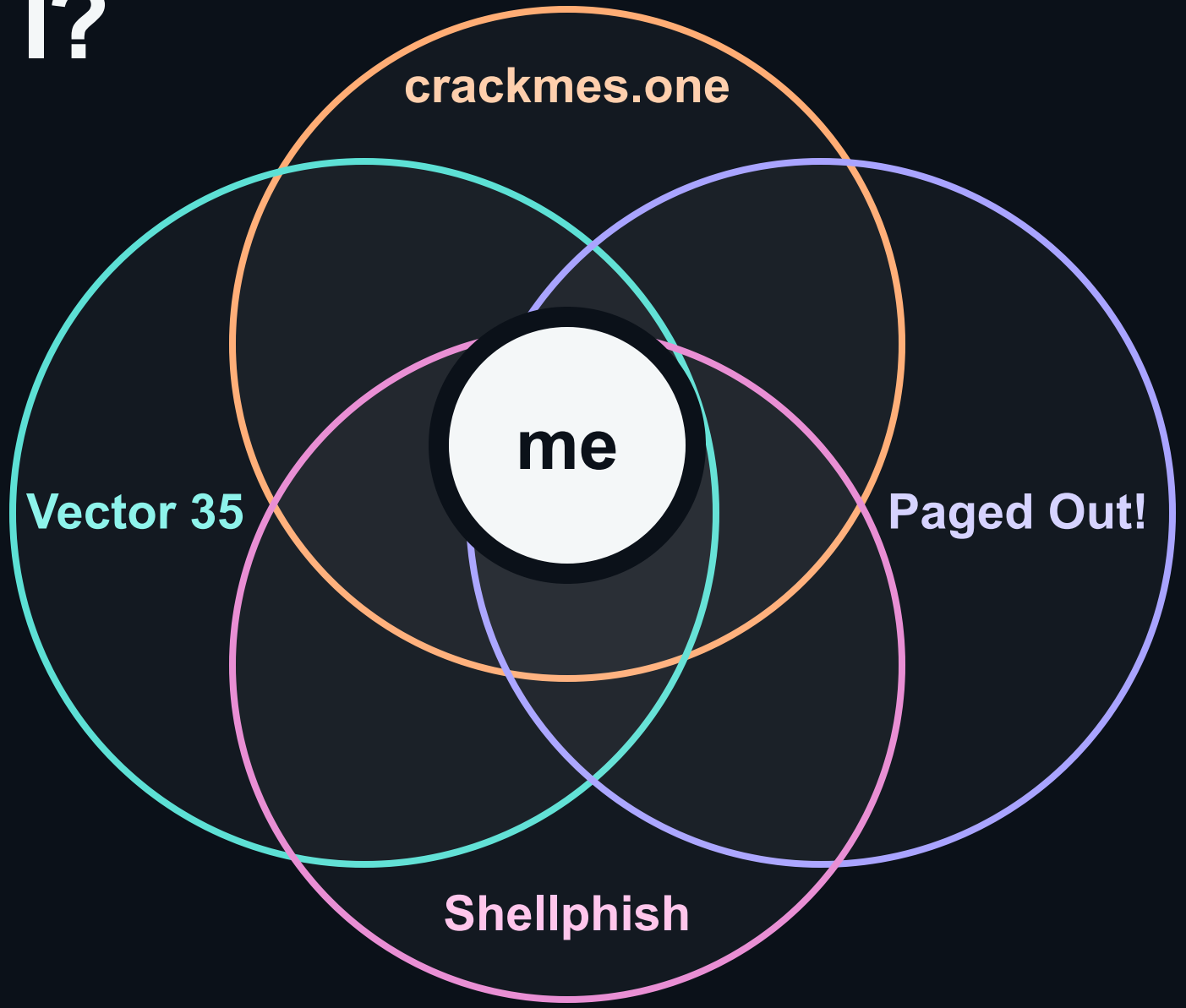
Bring Your Own Trusted Caller

A New Way to Exploit Vulnerable Windows Drivers

Xusheng Li | Vector 35

Who am I?



I am a…

- Software developer
- Reverse engineer
- Vulnerability researcher
- Malware analyst
- CTF player

Recap:

Bring Your Own Vulnerable Driver (BYOVD)

Bring Your Own Vulnerable Driver (1)

An attacker with local admin rights loads a legitimate, signed but vulnerable Windows kernel driver.

Then abuses it for kernel-level control—often to disable security software.

Definition adapted from Picus Security · May 2026

Bring Your Own Vulnerable Driver (2)

1. **Compromise a Windows host**
   phishing or exploitation
2. **Run in user mode**
   standard user or administrator
3. **Seek kernel-level capability**

Bring Your Own Vulnerable Driver (3)

What can kernel-level access enable?

Terminate AV / EDR processes

Access privileged information

Change kernel state

Bring Your Own Vulnerable Driver (4)

Windows checks kernel-driver signatures.

> **An unsigned attacker-built driver usually cannot load.**

Microsoft · Driver Signing Policy

Bring Your Own Vulnerable Driver (5)

Some legitimate signed drivers already have the capability.

ANTIVIRUS APP

Requests a process stop

IOCTL →

SIGNED DRIVER

Terminates the process

Bring Your Own Vulnerable Driver (6)

Bring the signed driver.
Call its vulnerable functionality.



BYOVD reports



CISCO TALOS · 2024

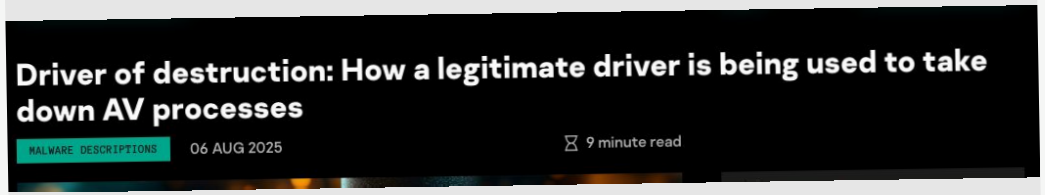
BlackByte brought four vulnerable drivers



UNIT 42 · 2024

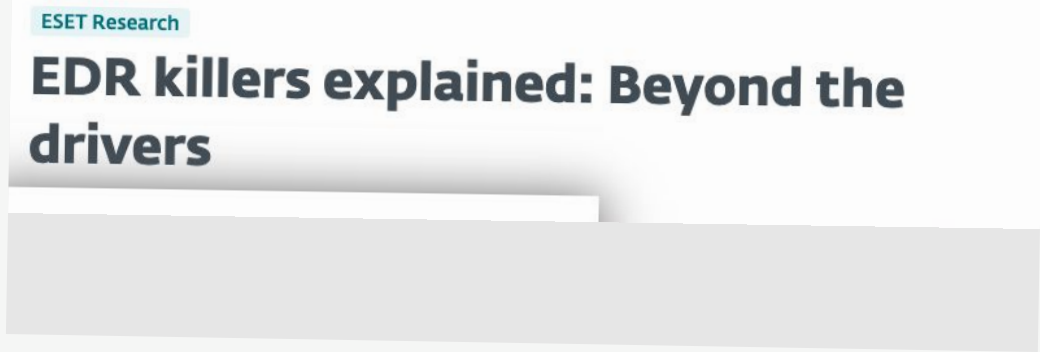
An extortion actor tested EDR evasion

More BYOVD reports



KASPERSKY · 2025

A signed driver used against AV processes



ESET RESEARCH · 2026

EDR killers are a repeatable tool class

Detecting BYOVD at scale

- **msFuzz**

  100 vulnerabilities found in 100 days

- **Unveiling BYOVD Threats: Malware's Use and Abuse of Kernel Drivers**

  NDSS 2026 · Analyzed malware loading 773 signed drivers

What do we do with a vulnerable driver?

01 **Microsoft vulnerable driver blocklist**

Report it to Microsoft. Confirmed risky versions may be blocked from loading.

02 **LOLDrivers**

Submit verified samples and hashes so defenders can hunt for them.

How do we fix the vulnerability?

01 · CODE DEFECT

Buffer or heap overflow?

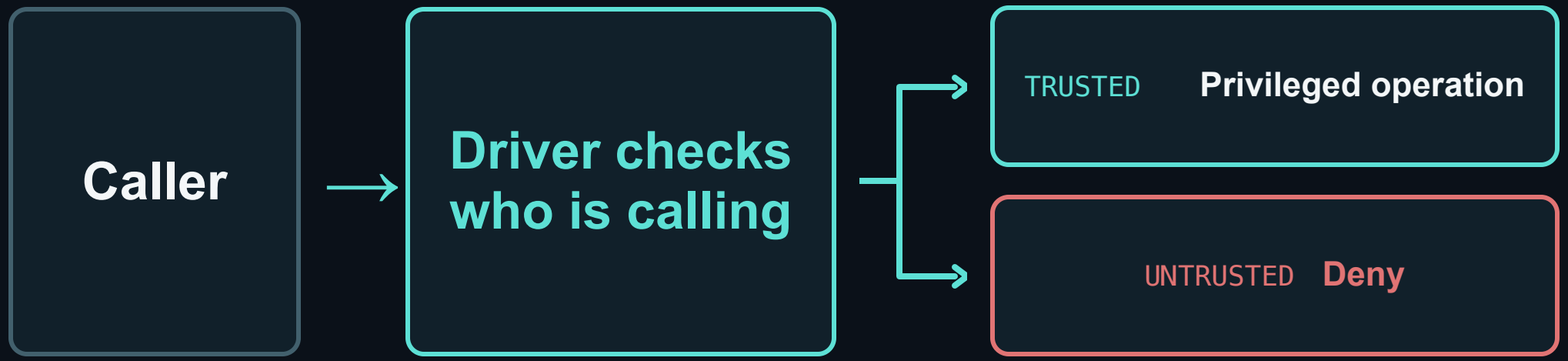
Repair the faulty code.

02 · ABUSED CAPABILITY

Process termination?

The driver may need this function.

A common fix: access control



Bring Your Own Trusted Caller (BYOTC)

Bring the client the driver already trusts.

Run your code inside that client.

The driver accepts its privileged request.

Malwarebytes Chameleon

USER-MODE CLIENT

MBAM.exe

Signed Malwarebytes app

→

KERNEL DRIVER

mbamchameleon.sys

Can terminate a process

How it normally works

01

Client registers

`IOCTL: 0x222008`

02

Driver checks signature

`Signed by Malwarebytes`

03

Client provides target PID

`IOCTL: 0x222024`

04

Driver terminates process

`ZwTerminateProcess(PID)`

Signature decides whether the driver acts

Signed MBAM.exe → Malwarebytes signature ✓ → **Trusted**

Unsigned client → No Malwarebytes signature ✕ → **Denied**

Where is the gap?

Signed executable `!=` trusted runtime code

Ways other code can enter a process

DLL loading

A loaded DLL may be untrusted.

Code injection

Another process can introduce code.

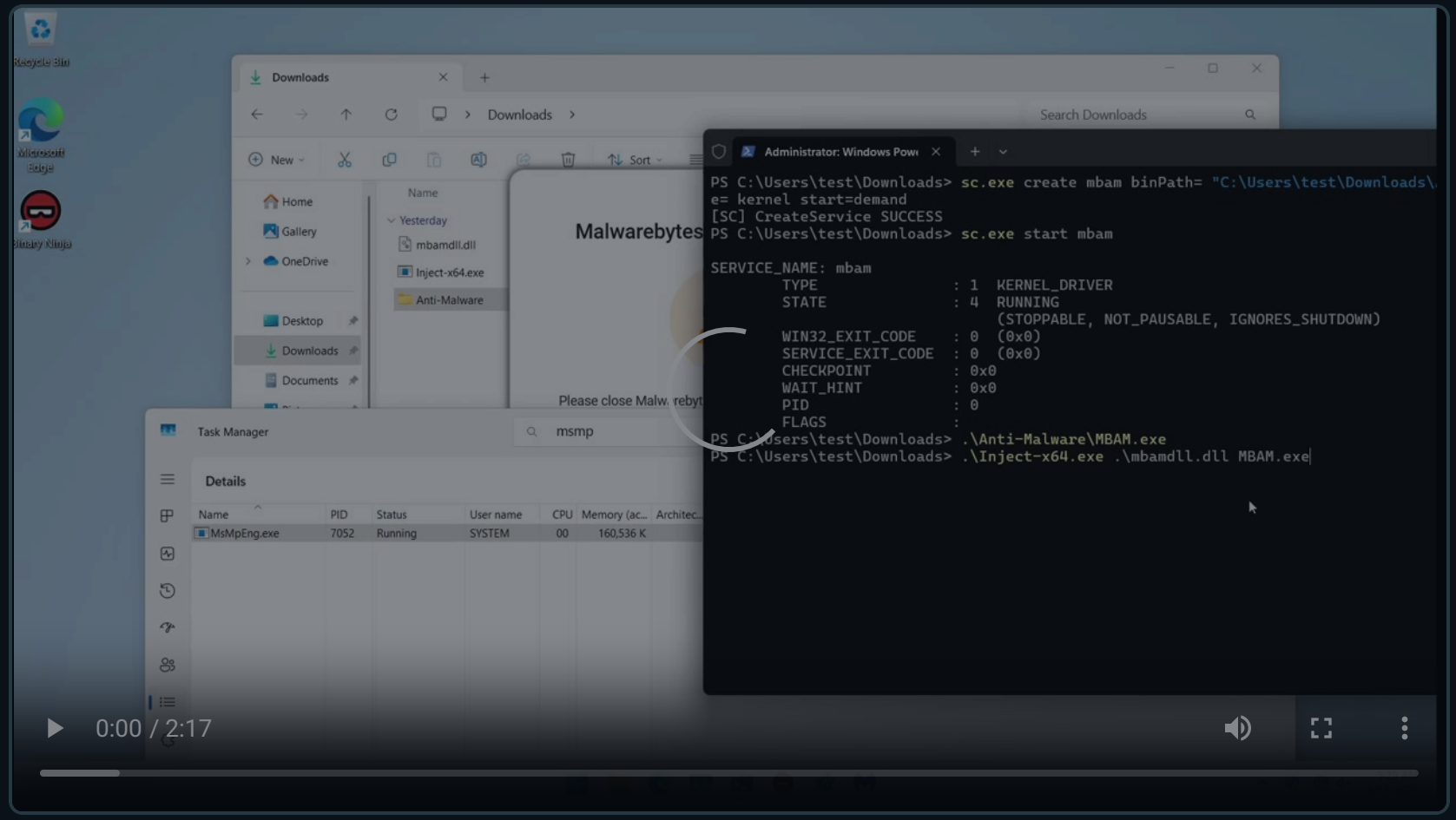
A signed image can run injected code

01 Launch genuine MBAM.exe

02 Inject a DLL into its process

03 Register, then request termination

Malwarebytes BYOTC demo



Malwarebytes disclosure timeline

Jun 15, 2025	**Reported to Malwarebytes**
Jul–Sep 2025	**Marked, then closed as duplicate**
Oct 31, 2025	**Fix reported in progress**
Sep 2026	**My retest: still reproducible**

Why no easy fix?

I don't blame Malwarebytes.

Reliable caller access control is hard.

We'll come back to this.

Checking More Drivers

Across antivirus and EDR products.

That's a *lot* of reverse engineering...

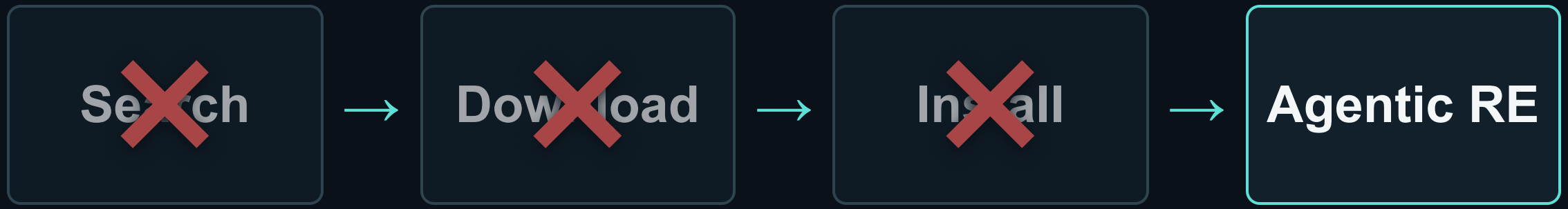
The solution is…



Analysis Pipeline

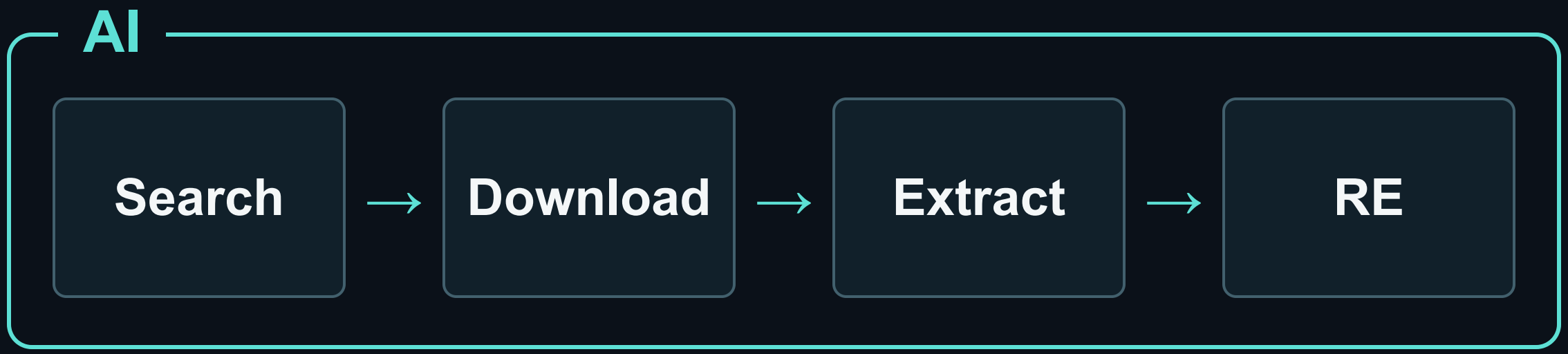


This isn’t the AI way



Why automate only the last step?

AI ran the whole pipeline



In Case You Want My Prompt

... find publicly available security products ...

... download their installers and extract the drivers

...

... analyze who can call each privileged operation

...

AI goes a long way

My M5 Mac's Windows on ARM VM can't load x64 kernel drivers.

AI handled the entire setup without my intervention

Downloaded x64 ISO → Built QEMU x64 VM → Installed Windows

The research so far

14

products screened

8

vulnerable drivers

6

accepted into LOLDrivers

The eight vulnerable drivers

Malwarebytes	accepted	Panda	accepted
Huorong	accepted	360 Total Security	accepted
Antiy ATool	accepted	Comodo	accepted
Avast	open	HitmanPro.Alert	open

How did the checks fail?

Signed EXE	The file is trusted; its running code can change.
Creator handles	A parent can alter a trusted child.
Signed callback	Avast's callback and stack check can be replayed.
Process name	A name is easy to copy.
Weak or no gate	Panda checks a public value; Comodo has no caller check.

Open Source

These tools protect access to their drivers.
Both protections can still be bypassed.

System Informer

VirtualBox

System Informer

A system management tool with a privileged driver.

Driver Classifies Clients into Five Trust Levels

MINIMUM	LOW	MEDIUM	HIGH	MAXIMUM

Process termination requires **MAXIMUM.**

Driver Protects the Client



Newly obtained process / thread handles lose memory-write and thread-control rights.

Be the Parent



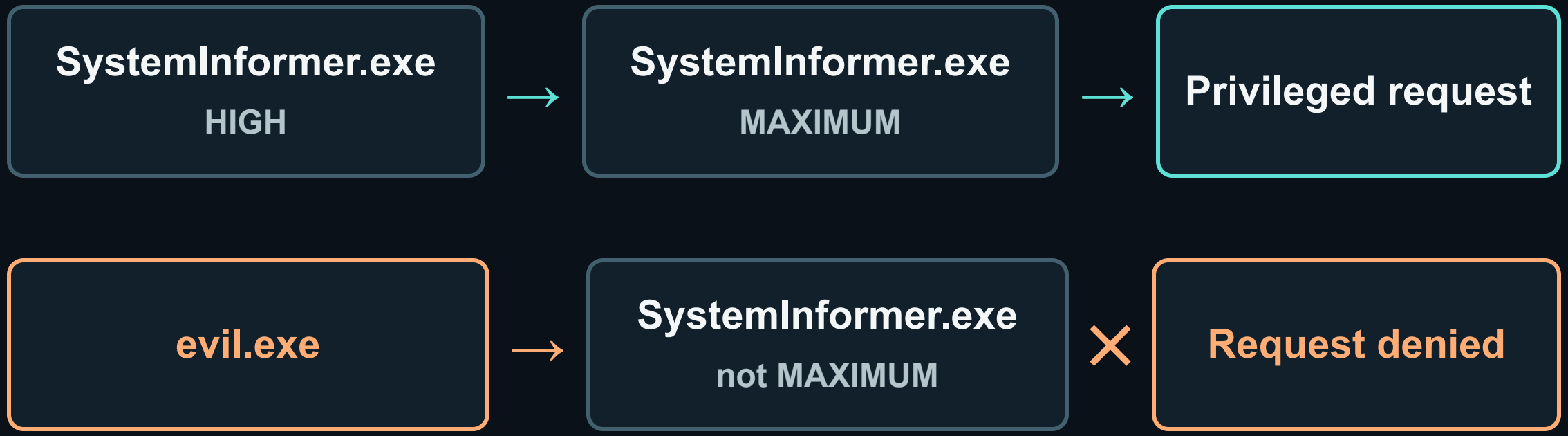
Parent receives PROCESS_ALL_ACCESS by default.

So the driver still permits:

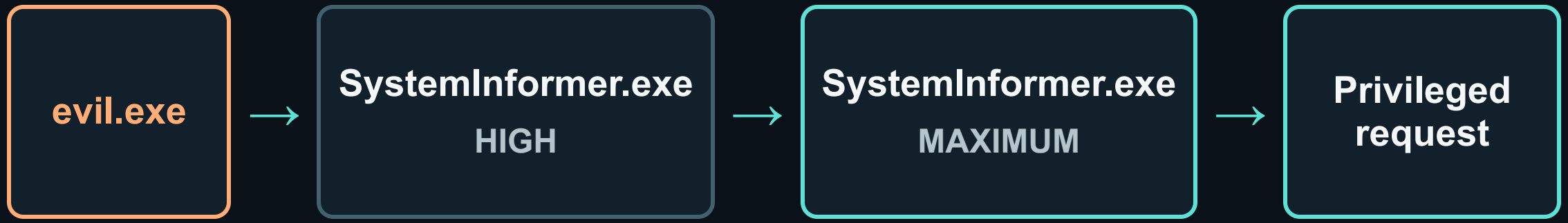
`PROCESS_VM_READ | PROCESS_VM_WRITE | PROCESS_VM_OPERATION`

Deny these rights, and System Informer may fail to launch.

MAXIMUM Requires a HIGH Parent



So be the grandparent



The Fix Relies on PPL



Windows enforces PPL protection around services.exe.

You can't easily inject code into a PPL process.

System Informer disclosure timeline

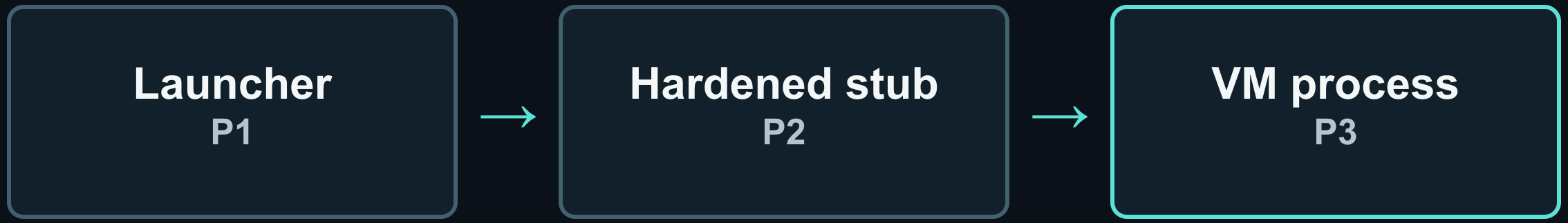
Apr 20, 2025	**Privately reported the issue**
Apr 21, 2025	**Maintainer proposed a fix**
Feb 1, 2026	**Rebased fix entered public history**
Aug 29, 2026	**Signed driver shipped in Release Channel**

VirtualBox Chooses a Different Design

Launch and prepare the child.

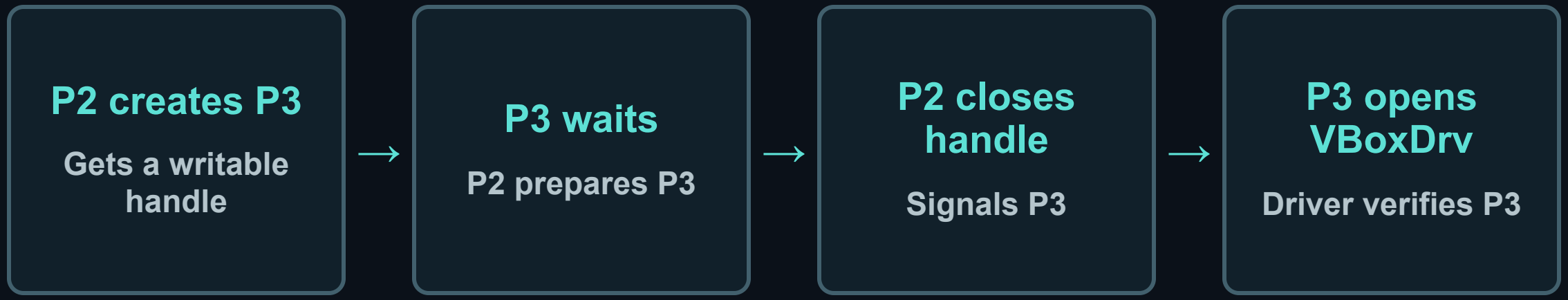
Then verify it for driver access.

VirtualBox Launches Three Processes



Only P3 receives the privileged driver session.

Preparation and Verification



The Driver Checks Live Handles

`SUPDrv-win.cpp:5441`
`supdrvNtProtectRestrictHandlesToProcessAndThread()`

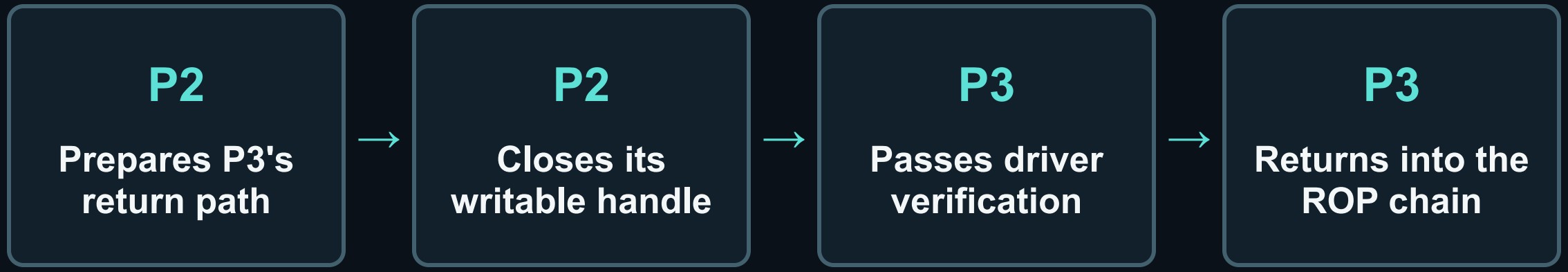
```
if (handle targets P3 &&
    rights include PROCESS_VM_WRITE)
    verification fails;
```

A writable parent handle must be gone before verification.

Why the Earlier Attack Fails Here

Keep a writable handle?	Verification fails.
Close it first?	No handle after trust.

A ROP Chain Across Verification



Code Reuse Bridges the Trust Check

We cannot carry the handle through.

We carry future control flow instead.

VirtualBox Disclosure Timeline

Jun 12, 2025	I reported the bypass
Dec 22, 2025	Six months later, Oracle disputed it
Jan 15, 2026	Oracle asked for a working PoC
Jan–Jun 2026	I was too busy to build the ROP chain
Jun 3, 2026	Oracle declined the report

How About Having AI Make the PoC?

But the bug was already patched in April 2026.

`bcfbf910`	Return-address checks
`6485c4ed`	Full stack-consistency checks

I was not acknowledged or notified.

Codex with Daybreak happily made the PoC. Too late for that route.

AI Found a Different Vulnerability

.rwxpg holds executable jump-back code in P3.

P2 alters that page, then closes its writable handle.

Verification checks the page's protection, but not its bytes.

P3 later executes the altered stub as a trusted driver client.

What About WESP?

Windows Endpoint Security Platform moves AV policy to user mode, with shared kernel enforcement.

Its normal AV client path requires Antimalware PPL and a WESP permission.

Utilities like System Informer and VirtualBox still have their own driver-client boundary.

Questions?

xusheng.dev/posts/byotc/main

xusheng@vector35.com

